UNIVERSIDAD

SIGLO 21

Taller de Construcción de Software

TRABAJO PRÁCTICO INTEGRADOR

SocioClub

Sistema web de gestión de socios, actividades e inscripciones para el Club Social y Deportivo Los Algarrobos (Córdoba)

Modalidad Grupal (4 o 5 integrantes)

Evaluación Exposiciones orales de avance + defensa + entrega final

Alcance Módulos 1 a 4 de la materia (16 lecturas)

Tecnologías HTML5 · CSS3 · JavaScript · Spring Boot · Thymeleaf · JPA/MySQL · API REST · React · Jest

1. Propósito del trabajo

Durante la materia se ven, lectura por lectura, piezas sueltas de una aplicación web: una página con HTML, CSS y JavaScript; un backend en capas con Spring Boot; persistencia con JPA y MySQL; una interfaz en React; una API REST; y pruebas. En este trabajo práctico van a **juntar todas esas piezas en un único sistema** que resuelve un problema real de un cliente real.

El objetivo no es solamente que el sistema funcione. El objetivo es que **cada integrante del grupo pueda explicar qué hizo, por qué lo hizo así y cómo viaja la información** desde que el usuario hace clic hasta que el dato se guarda en la base y vuelve a la pantalla.

> **En una frase**
>
> Van a construir para un club de barrio de Córdoba un **sitio público**, un **sistema interno para la secretaría** y una **app web para los socios**, todo conectado a la misma base de datos, y lo van a defender oralmente.

2. El caso: Club Social y Deportivo Los Algarrobos

El Club Social y Deportivo Los Algarrobos es un club de barrio de la zona norte de la ciudad de Córdoba. Tiene alrededor de **650 socios** y ofrece **8 actividades** deportivas y recreativas para chicos y adultos: fútbol infantil, vóley, básquet, natación, patín artístico, ajedrez, funcional y yoga.

Hoy toda la gestión se hace a mano. Marta, la secretaria, anota las inscripciones en un cuaderno y después las pasa a una planilla de Excel. Los horarios se comunican por un grupo de WhatsApp y las consultas llegan por teléfono o por mensaje directo a los profesores. Esto genera problemas concretos que la Comisión Directiva quiere resolver:

- **Cupos excedidos:** en marzo se anotaron 52 chicos en fútbol infantil cuando la cancha y los profesores alcanzan para 40.
- **Inscripciones duplicadas:** un mismo socio aparece dos veces en la misma actividad porque se anotó en secretaría y también le avisó al profesor.
- **Edades fuera de rango:** chicos de 14 años inscriptos en la categoría de 6 a 12.
- **Socios con cuota impaga** que siguen inscribiéndose a nuevas actividades sin que nadie lo detecte.
- **Falta de información pública:** los vecinos no saben qué actividades hay, en qué horario ni cuánto cuestan, y saturan la secretaría con consultas.

> **Lo que pidió la Comisión Directiva**
>
> “Queremos tres cosas. Primero, una página del club donde cualquier vecino vea las actividades, los horarios y los precios, y pueda dejar una preinscripción. Segundo, un sistema para que Marta cargue socios y actividades e inscriba gente sin que se nos pasen los cupos ni las edades. Y tercero, que cada socio pueda entrar desde el celular con su DNI, ver a qué está anotado, cuánto paga y anotarse solo.”
>
> — Presidente del Club Los Algarrobos

3. Requerimientos del cliente

3.1 Requerimientos funcionales

Código	Requerimiento	Lo usa
RF1	Mostrar públicamente las actividades del club con día, horario, edades, profesor y cuota mensual.	Vecinos
RF2	Permitir filtrar las actividades por disciplina y por edad.	Vecinos / Socios
RF3	Recibir preinscripciones desde un formulario web validado.	Vecinos
RF4	Alta, modificación, baja y listado de actividades.	Secretaría
RF5	Alta, modificación y listado de socios, con búsqueda por apellido y listado paginado.	Secretaría
RF6	Inscribir un socio a una actividad aplicando las reglas de negocio (sección 3.2).	Secretaría / Socios
RF7	Ver los inscriptos activos de cada actividad y su cupo disponible.	Secretaría
RF8	Ingreso del socio a la app con su DNI y consulta de sus inscripciones.	Socios
RF9	Darse de baja de una actividad (baja lógica: la inscripción queda en estado BAJA).	Socios / Secretaría
RF10	Mostrar al socio su cuota mensual estimada según las actividades a las que está inscripto.	Socios

3.2 Reglas de negocio

Estas reglas son el corazón del sistema. Se implementan en la **capa de servicio** del backend (no en el controlador ni en la vista), salvo RN7 que se implementa en JavaScript del lado del cliente.

Código	Regla	Qué debe pasar si no se cumple
RN1	**Cupo:** una actividad no puede tener más inscripciones ACTIVAS que su cupo máximo.	Se rechaza la inscripción con el mensaje “La actividad no tiene cupo disponible”.
RN2	**Sin duplicados:** un socio no puede tener dos inscripciones ACTIVAS en la misma actividad.	Se rechaza con “El socio ya está inscripto en esta actividad”.
RN3	**Edad:** la edad del socio (calculada a la fecha de hoy desde su fecha de nacimiento) debe estar entre la edad mínima y máxima de la actividad.	Se rechaza indicando el rango permitido.
RN4	**Cuota al día:** un socio con estado de cuota MOROSO no puede inscribirse a nuevas actividades (sí puede consultar y darse de baja).	Se rechaza con “Debe regularizar su cuota en secretaría”.
RN5	**Máximo 3 actividades:** un socio puede tener como máximo 3 inscripciones ACTIVAS.	Se rechaza con “Alcanzó el máximo de 3 actividades”.
RN6	**Bonificación:** la cuota mensual estimada es la suma de las cuotas de sus actividades activas, con 10 % de descuento si tiene 2 actividades y 15 % si tiene 3.	Se calcula en una función JavaScript y se prueba con Jest.
RN7	**DNI válido:** el DNI debe tener 7 u 8 dígitos numéricos, sin puntos.	El formulario no se envía y muestra el error junto al campo.

3.3 Datos iniciales del club

Para que todos los grupos trabajen sobre la misma realidad, la base de datos debe arrancar con estas actividades (pueden cargarlas con un script SQL o un archivo data.sql):

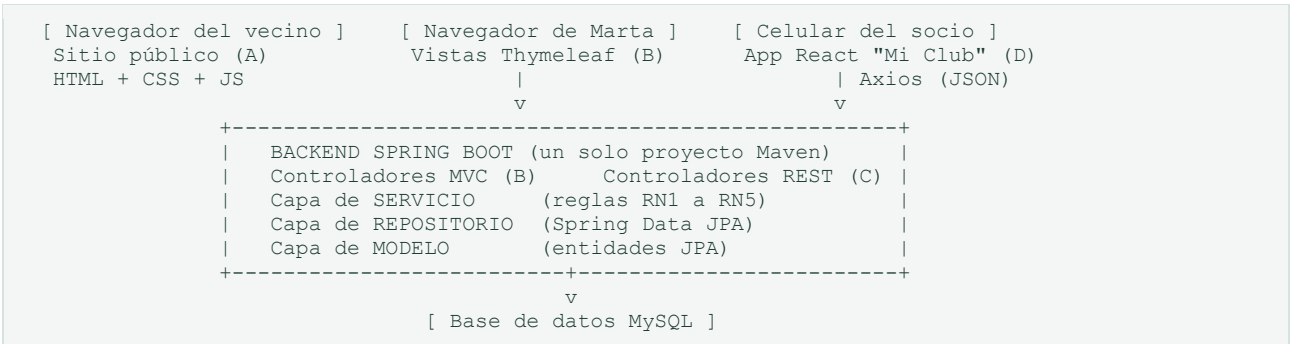
Actividad	Disciplina	Días y horario	Edades	Cupo	Cuota	Profesor/a
Fútbol infantil	Fútbol	Lun y Mié 18:00	6 a 12	40	$ 18.000	Diego Ferreyra
Vóley mixto	Vóley	Mar y Jue 19:00	13 a 60	24	$ 20.000	Laura Ceballos
Básquet juvenil	Básquet	Lun y Vie 19:30	12 a 17	20	$ 19.000	Martín Oviedo
Natación	Natación	Mar y Jue 17:00	5 a 70	30	$ 26.000	Sofía Bustos
Patín artístico	Patín	Mié y Vie 17:30	5 a 15	25	$ 21.000	Carla Peralta
Ajedrez	Ajedrez	Sáb 10:00	8 a 99	16	$ 9.000	Raúl Giménez
Funcional adultos	Fitness	Lun, Mié y Vie 20:30	18 a 65	30	$ 22.000	Paula Luna
Yoga	Bienestar	Mar y Jue 9:00	16 a 99	15	$ 17.000	Inés Arguello

Además deben cargar **al menos 15 socios de prueba** con edades variadas, incluyendo **por lo menos 2 socios MOROSOS** y una actividad con el **cupo casi completo**, para poder demostrar todas las reglas en la exposición.

4. La solución que van a construir

El sistema tiene **tres caras** que comparten una sola base de datos MySQL y un solo backend Spring Boot:

Componente	Para quién	Tecnología	Módulo de la materia
A. Sitio público del club	Vecinos	HTML5 semántico, CSS3 responsivo, JavaScript (DOM y eventos)	Módulo 1 (Lecturas 1 a 4)
B. Sistema de secretaría (backoffice)	Marta y la Comisión	Spring Boot + Spring MVC + Thymeleaf + Spring Data JPA + MySQL	Módulo 2 (Lecturas 1 a 4)
C. API REST	La app de socios	Spring Boot @RestController, JSON, probada con Postman	Módulo 4 (Lecturas 1 y 2)
D. App “Mi Club” para socios	Socios	React (componentes, props, estado, hooks, JSX), React Router y Axios	Módulo 3 (Lecturas 1 a 4) y Módulo 4 (Lectura 3)
E. Pruebas	El equipo	Jest, React Testing Library y colección de Postman	Módulo 4 (Lectura 4)



Idea clave de arquitectura

Los controladores de Thymeleaf (B) y los controladores REST (C) **usan la misma capa de servicio**. Las reglas de negocio se escriben **una sola vez**. Si una inscripción entra por el sistema de Marta o por la app del socio, se valida exactamente igual. Esto lo van a tener que explicar en la defensa.

5. Partes del trabajo y requisitos mínimos

Cada parte indica qué tienen que construir, qué conceptos de la materia deben aparecer sí o sí, y qué van a tener que poder explicar. Los requisitos son **mínimos**: pueden agregar más, siempre que lo puedan defender.

Parte A — Sitio público del club (Módulo 1)

Qué construir

- Tres páginas como mínimo: **index.html** (presentación del club, historia breve, ubicación), **actividades.html** (catálogo) y **preinscripcion.html** (formulario).
- Un único archivo **estilos.css** externo y un único archivo **script.js** externo (o uno por página), vinculados correctamente.

Requisitos de la materia

- ☐ Estructura HTML5 semántica: header, nav, main, section, article, aside y footer usados con sentido, no por decorar.
- ☐ Encabezados jerárquicos, listas, imágenes con atributo alt, enlaces entre páginas y meta viewport.
- ☐ CSS en hoja externa, usando selectores de tipo, de clase, de ID y al menos un selector de atributo.
- ☐ Diseño responsivo con al menos una media query (en celular las tarjetas de actividades se ven en una columna).
- ☐ El catálogo de actividades se genera con JavaScript a partir de un arreglo de objetos, usando una **clase Actividad** con al menos un método (por ejemplo esAptaParaEdad(edad)).
- ☐ Filtro por disciplina y por edad que actualiza la página manipulando el DOM, con eventos registrados mediante **addEventListener**.
- ☐ Formulario de preinscripción validado con JavaScript (campos obligatorios, email, DNI según RN7). Los errores se muestran en la página junto a cada campo, **no con alert()**.
- ☐ Uso justificado de al menos un objeto del navegador visto en la Lectura 4 (window, location, history, navigator o screen). Ejemplo: botón “Volver” con history.back().

Lo que tienen que poder explicar

- Qué es el DOM y cómo su código encuentra y modifica un elemento.
- Qué pasa, paso a paso, desde que el usuario cambia el filtro hasta que la lista se redibuja.
- Por qué un elemento usa una clase CSS y otro un ID; qué hace su media query.

Parte B — Sistema de secretaría con Spring Boot y Thymeleaf (Módulo 2)

Qué construir

- Un proyecto **Maven** generado con Spring Initializr con las dependencias Spring Web, Thymeleaf, Spring Data JPA y MySQL Driver.
- Pantallas Thymeleaf para: listado, alta, edición y baja de actividades (con cupo disponible); listado paginado de socios con búsqueda por apellido; alta y edición de socios; formulario de inscripción (elegir socio y actividad); listado de inscriptos de una actividad.

Requisitos de la materia

- ☐ Arquitectura en capas con paquetes separados: **model**, **repository**, **service** y **controller**. Ninguna vista ni controlador accede directo al repositorio.

- ☐ Tres entidades JPA como mínimo: **Socio**, **Actividad** e **Inscripcion** (ver Anexo B), con @Entity, @Id, @GeneratedValue, @Column y las relaciones @ManyToOne / @OneToMany con @JoinColumn y mappedBy.
- ☐ Repositorios que extienden **JpaRepository**, con al menos dos **métodos derivados del nombre** (por ejemplo findByApellidoContainingIgnoreCase, findByDisciplina) y al menos una consulta con **@Query** (por ejemplo contar inscripciones activas de una actividad).
- ☐ Paginación del listado de socios usando **Pageable**.
- ☐ Reglas RN1 a RN5 implementadas en la clase de servicio de inscripciones. Si una regla no se cumple, la vista muestra el mensaje correspondiente.
- ☐ Vistas Thymeleaf con th:each, th:text, th:if, th:object / th:field y **fragmentos reutilizables** (encabezado y pie comunes a todas las pantallas).
- ☐ Conexión a MySQL configurada en **application.properties** y datos iniciales de la sección 3.3 cargados.

Lo que tienen que poder explicar

- Qué hace cada capa y por qué la regla del cupo está en el servicio y no en el controlador.
- Qué es un ORM, qué es una entidad y cómo se traduce la relación Socio–Inscripcion–Actividad en tablas.
- Cómo Spring “arma” la consulta SQL a partir del nombre de un método del repositorio.
- Qué es un bean y qué significa que Spring inyecte el servicio en el controlador.

Parte C — API REST (Módulo 4, Lecturas 1 y 2)

Qué construir

- Dentro del mismo proyecto Spring Boot, controladores con **@RestController** bajo la ruta **/api** que expongan los endpoints mínimos del **Anexo A**, reutilizando la capa de servicio de la Parte B.
- Una **colección de Postman** con todos los endpoints, incluyendo los casos de error (sin cupo, duplicado, moroso, edad fuera de rango, máximo de actividades).

Requisitos de la materia

- ☐ Uso correcto de los verbos HTTP: GET para consultar, POST para crear, PUT para modificar y DELETE para eliminar.
- ☐ Anotaciones @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PathVariable y @RequestBody.
- ☐ Respuestas en JSON con códigos de estado coherentes: 200 OK, 201 Created, 204 No Content, 404 Not Found y 409 Conflict cuando una regla de negocio rechaza la operación (con el mensaje de la regla en el cuerpo).
- ☐ Configuración de **CORS** en el backend para permitir las peticiones de la app React (localhost:3000).

Lo que tienen que poder explicar

- Qué hace que su API sea REST (recursos, verbos, sin estado, representación JSON) y en qué se diferencia de SOAP.
- Por qué el navegador bloquea la petición si no configuran CORS.
- Cómo probaron cada endpoint en Postman y qué devuelve cada caso de error.

Parte D — App “Mi Club” para socios en React (Módulos 3 y 4)

Qué construir

- Un proyecto React (creado con create-react-app como en las lecturas) con cuatro rutas usando **React Router (BrowserRouter)**:
  - **/** — Ingreso: el socio escribe su DNI y la app lo busca en la API.
  - **/actividades** — Catálogo con filtro por disciplina, indicando el cupo disponible.
  - **/actividades/:id** — Detalle de la actividad con el botón “Inscribirme”.
  - **/mis-actividades** — Inscripciones activas del socio, botón “Darme de baja” y cuota mensual estimada (RN6).

Requisitos de la materia

- ☐ Solo componentes funcionales, con nombres en PascalCase y funciones en camelCase, según la nomenclatura de la Lectura 1 del Módulo 3.
- ☐ Al menos estos componentes: Encabezado, Pie, TarjetaActividad, ListaActividades, DetalleActividad, MisActividades y ResumenCuota.
- ☐ Datos que fluyen de padre a hijo con **props**, validadas con **PropTypes** en TarjetaActividad y ResumenCuota.
- ☐ Estado local con **useState** y carga de datos con **useEffect**.
- ☐ El socio que ingresó se comparte entre pantallas con **createContext / useContext** (no se pasa por props de componente en componente).
- ☐ JSX con listas renderizadas con map y atributo **key**, y **renderizado condicional**: etiqueta “Sin cupo” y botón deshabilitado cuando no hay lugar; mensaje cuando el socio no tiene actividades.
- ☐ Estilos con archivos CSS externos y al menos una **clase aplicada dinámicamente** según el estado (por ejemplo, tarjeta en gris cuando no hay cupo).
- ☐ Toda la comunicación con el backend centralizada en un archivo de servicio (por ejemplo src/servicios/api.js) usando **Axios**. Los mensajes de error que devuelve la API (409) se muestran al socio.

Lo que tienen que poder explicar

- Qué es el DOM virtual y por qué React vuelve a dibujar un componente cuando cambia su estado.
- Diferencia entre props y estado; para qué usaron useEffect y qué pasa con su arreglo de dependencias.
- Por qué el socio logueado está en un contexto y no en props.
- El recorrido completo de una inscripción: clic en “Inscribirme” → Axios → controlador REST → servicio → repositorio → MySQL → respuesta → actualización de la pantalla.

Parte E — Pruebas (Módulo 4, Lectura 4)

- ☐ **Pruebas unitarias con Jest** (mínimo 6) sobre dos funciones puras en src/utils: calcularCuotaMensual(actividades) (RN6) y esDniValido(dni) (RN7). Deben cubrir casos normales y bordes: 0, 1, 2 y 3 actividades; DNI con puntos, con letras, con 6 y con 9 dígitos.
- ☐ **Pruebas de componente con React Testing Library** (mínimo 2) sobre TarjetaActividad, usando render, screen y fireEvent: que muestre el nombre de la actividad, y que con cupo 0 muestre “Sin cupo” y el botón esté deshabilitado.
- ☐ **Pruebas de la API** con la colección de Postman (Parte C).
- ☐ **Prueba de sistema documentada:** una tabla con 6 casos de punta a punta (caso, pasos, resultado esperado, resultado obtenido), que incluya al menos un caso exitoso y uno por cada regla RN1 a RN5.

Parte F — Documentación y bitácora de IA

- **README.md** en el repositorio con: integrantes, cómo crear la base, cómo levantar el backend y la app React, y usuarios/DNI de prueba.
- **Informe técnico** en PDF (entre 8 y 12 páginas) con: descripción de la solución, diagrama de arquitectura, modelo de datos, tabla de endpoints, capturas de las tres caras del sistema, resultados de las pruebas y conclusiones del equipo.
- **Bitácora de uso de IA** (formato en el Anexo D), como último capítulo del informe.

6. Hitos y exposiciones orales

En las **tres clases que hay entre el primer y el segundo parcial** cada grupo expone **una vez**. En cada clase exponen aproximadamente 5 grupos. El orden se define por sorteo en la clase de presentación de este trabajo.

Como no todos exponen en el mismo momento del cuatrimestre, **lo que se exige en la exposición depende de la semana que les tocó**. Se evalúa contra el hito de esa semana, así nadie queda en desventaja. La **entrega final completa es obligatoria para todos los grupos** al terminar la materia.

Momento	Qué deben mostrar funcionando como mínimo	Estado esperado
Semana 1 (1.ª clase después del primer parcial)	• Parte A completa: sitio público responsivo con filtro y formulario validado. • Parte B: proyecto Maven en capas, entidades y repositorios creados, base MySQL con datos iniciales y ABM de actividades en Thymeleaf funcionando. • Plan de trabajo para lo que falta.	**Avance (aprox. 40 %)**
Semana 2 (2.ª clase)	• Todo lo de la semana 1. • Parte B completa: socios con paginación y búsqueda; inscripciones con reglas RN1 a RN5. • Parte D iniciada: app React con rutas y componentes funcionando con datos locales (un arreglo en el código), usando props, estado y contexto.	**Avance (aprox. 70 %)**
Semana 3 (3.ª clase)	• Todo lo anterior. • Parte C: API REST completa y probada en Postman. • Parte D conectada a la API con Axios: ingreso por DNI, inscripción y baja reales. • Parte E iniciada (al menos las pruebas con Jest).	**Casi final (aprox. 90 %)**
Entrega final (última clase de la materia)	• Partes A a F completas, incorporando las devoluciones recibidas en la exposición.	**Final (100 %)**

6.1 Formato de la exposición

Momento	Duración	Qué pasa
Presentación	2 min	Quiénes son, qué parte hizo cada uno, en qué estado está el proyecto respecto del hito.
Demostración en vivo	7 min	El sistema funcionando, no capturas. Deben mostrar al menos una regla de negocio rechazando una operación.
Recorrido técnico	3 min	Muestran el código de una funcionalidad elegida por el docente en el momento y explican cómo viaja la información entre capas.

Momento	Duración	Qué pasa
Defensa individual	6 min	El docente pregunta a integrantes elegidos por él, sobre cualquier parte del proyecto, no solo sobre la que hizo cada uno.
Devolución	2 min	Observaciones del docente para incorporar en la entrega final.

Reglas de la exposición

- Exponen **todos los integrantes**. Quien no esté presente sin justificación no tiene nota de exposición ni de defensa.
- El proyecto debe **correr en la computadora del grupo** antes de que empiece su turno (backend, base y app levantados).
- Se pide tener el **repositorio abierto** en el editor: el docente puede pedir ver cualquier archivo.
- El docente puede pedir un **cambio pequeño en vivo** (por ejemplo, modificar el máximo de actividades de 3 a 4, o agregar un dato a la tarjeta de actividad) para verificar que entienden dónde está cada cosa.
- Tiempo máximo por grupo: **20/25 minutos**.

7. Uso de inteligencia artificial

Pueden y van a usar herramientas de IA (ChatGPT, Claude, Copilot, Gemini u otras) para hacer este trabajo. Está **permitido** y es parte de cómo se trabaja hoy en la industria. Lo que se evalúa no es si lo escribieron a mano, sino si **entienden, controlan y pueden defender** lo que entregan.

Las reglas son tres

1. Ustedes son responsables de todo el código. Si está en el repositorio, cualquier integrante tiene que poder explicarlo. “Lo hizo la IA” no es una respuesta válida en la defensa.

2. El código tiene que respetar lo que se vio en la materia. La IA suele proponer otras herramientas (Vite en lugar de create-react-app, fetch en lugar de Axios, Lombok, Spring Security, Redux Toolkit, Tailwind, etc.). Si las usan y no las pueden explicar ni justificar, se considera un incumplimiento del requisito.

3. Documentan cómo la usaron en la bitácora de IA (Anexo D): qué le pidieron, qué les dio, qué estaba mal o no servía y cómo lo corrigieron. Se piden **al menos 5 entradas**, y en **al menos 3** tiene que haber un error o ajuste que el grupo detectó.

Consejo práctico: después de que la IA les genere algo, pídanle que se lo explique línea por línea, y después explíquenselo ustedes a un compañero sin mirar. Si no pueden, no lo entienden todavía.

8. Entrega final

Qué	Formato	Detalle
Repositorio	GitHub (público o con el docente invitado)	Carpetas /sitio-publico, /backend y /app-socios en el mismo repositorio. Commits de todos los integrantes a lo largo del cuatrimestre.
Base de datos	Archivo .sql	Script de creación y datos iniciales (sección 3.3 + socios de prueba).
Colección de Postman	Archivo .json exportado	Todos los endpoints del Anexo A con casos exitosos y de error.

Qué	Formato	Detalle
Informe técnico	PDF, 8 a 12 páginas	Según Parte F, incluyendo la prueba de sistema y la bitácora de IA.
README.md	En la raíz del repositorio	Instrucciones para que el docente pueda levantar el proyecto.

Fecha: última clase de la materia. **Medio:** se sube por el aula virtual un único archivo PDF (el informe) que en la primera página tenga el enlace al repositorio. **Nombre del archivo:** TPI_SocioClub_GrupoN.pdf (N = número de grupo).

9. Evaluación

9.1 Rúbrica del producto final (100 puntos)

Criterio	Pts.	Excelente	Suficiente	Insuficiente
A. Sitio público	10	Semántico, responsivo, catálogo y filtro con clase JS y DOM, validación completa sin alert.	Funciona, con algún requisito parcial.	Faltan páginas, no es responsivo o el catálogo está escrito a mano.
B. Backoffice Spring + Thymeleaf + JPA	25	Capas limpias, relaciones JPA correctas, métodos derivados, @Query, paginación, RN1–RN5 en el servicio, fragmentos.	Funciona con capas, pero falta alguna regla o requisito.	Lógica en el controlador o la vista, reglas ausentes, no persiste en MySQL.
C. API REST	15	Endpoints completos, verbos y códigos correctos, CORS, colección Postman con errores.	Endpoints principales funcionando, códigos genéricos.	Faltan endpoints o no reutiliza el servicio.
D. App React	25	Rutas, props con PropTypes, useState/useEffect, contexto, renderizado condicional, Axios centralizado, errores visibles.	Funciona con la API pero falta algún requisito.	No consume la API o no usa los conceptos de la materia.
E. Pruebas	10	6+ pruebas Jest, 2+ de componente, Postman y prueba de sistema completa.	Pruebas presentes pero incompletas.	Sin pruebas o no corren.
F. Documentación y bitácora IA	15	README que permite levantar todo, informe claro, bitácora con correcciones reales.	Documentación correcta pero escueta.	No se puede levantar el proyecto o falta la bitácora.

10. Condiciones generales

- Grupos de **4 o 5 integrantes**, conformados en la clase de presentación. Los cambios de grupo posteriores se acuerdan con el docente.
- Los grupos pueden intercambiar ideas, pero el código y el informe son propios. Dos entregas sustancialmente iguales se consideran copia para ambos grupos.

- Pueden agregar funcionalidades extra (por ejemplo, un panel con estadísticas de ocupación), siempre que las cumplan con las tecnologías de la materia y las puedan defender. Los extras no compensan requisitos mínimos faltantes.

Anexo A — Endpoints mínimos de la API

Verbo	Ruta	Qué hace	Respuestas
GET	/api/actividades	Lista todas las actividades con su cupo disponible. Acepta ?disciplina=Fútbol	200
GET	/api/actividades/{id}	Detalle de una actividad.	200 / 404
POST	/api/actividades	Crea una actividad.	201
PUT	/api/actividades/{id}	Modifica una actividad.	200 / 404
DELETE	/api/actividades/{id}	Elimina una actividad sin inscriptos activos.	204 / 404 / 409
GET	/api/socios/dni/{dni}	Busca un socio por DNI (ingreso a la app).	200 / 404
GET	/api/socios/{id}/inscripciones	Inscripciones ACTIVAS de un socio.	200 / 404
POST	/api/inscripciones	Inscribe un socio. Cuerpo: { "socioId": 3, "actividadId": 1 }. Aplica RN1 a RN5.	201 / 404 / 409
PUT	/api/inscripciones/{id}/baja	Baja lógica: cambia el estado a BAJA.	200 / 404

Ejemplo de respuesta cuando una regla rechaza la inscripción (409 Conflict):

```
{
  "error": "La actividad no tiene cupo disponible",
  "regla": "RN1"
}
```

Anexo B — Modelo de datos sugerido

Entidad	Atributos	Relaciones
Socio	id, dni (único), nombre, apellido, fechaNacimiento, email, telefono, estadoCuota (AL_DIA / MOROSO)	Un socio tiene muchas inscripciones (@OneToMany, mappedBy = "socio").
Actividad	id, nombre, disciplina, profesor, dias, horario, edadMinima, edadMaxima, cupoMaximo, cuotaMensual	Una actividad tiene muchas inscripciones (@OneToMany, mappedBy = "actividad").
Inscripcion	id, fechaInscripcion, estado (ACTIVA / BAJA)	Muchas inscripciones pertenecen a un socio y a una actividad (@ManyToOne + @JoinColumn).

> **Desafío esperable**
>
> Cuando conviertan las entidades a JSON en la API, la relación de ida y vuelta Socio ↔ Inscripcion puede generar un ciclo infinito. Tienen que detectarlo, resolverlo y **explicar cómo lo resolvieron**. Es una pregunta segura en la defensa.

Anexo C — Formato de la bitácora de IA

N.º	Parte	Qué le pedimos a la IA (resumen del prompt)	Qué nos devolvió	Qué estaba mal / qué cambiamos y por qué
1	B	Ej.: “Generá la entidad Inscripcion con relación a Socio y Actividad usando JPA.”	La entidad con @ManyToOne y Lombok.	Sacamos Lombok porque no se vio en la materia y escribimos getters y setters. Agregamos el estado ACTIVA/BAJA que no había incluido.
2				
3				
4				
5				